SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Sairaj C	**Reg. No.:**	192411018
Section / Batch:	BE-CSE	**Date:**	28/07/2026

Debugging Bubble Sort Code

The following Bubble Sort implementation does not correctly sort the array and performs unnecessary iterations. Identify logical errors in loop conditions and swapping mechanism, and explain their causes. Debug the code step-by-step to ensure correct sorting. Optimize the algorithm by reducing unnecessary passes using a flag. Analyze the time complexity of the corrected version. Rewrite the final optimized code with proper comments.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-1):
            if arr[j] < arr[j+1]:   # ERROR
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

```
def bubble_sort(arr):
    n = len(arr)

    # Traverse through all elements of the array
    for i in range(n):
        # Flag to check if any swapping occurs
        swapped = False

        # Last i elements are already sorted, so no need to check them
        for j in range(0, n - i - 1):

            # Compare adjacent elements
            # Swap them if they are in the wrong order
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swapping occurred in this pass,
        # the array is already sorted
        if not swapped:
```

```
Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]

=== Code Execution Successful ===
```

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I Sairaj C(192411018) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sairaj C

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of	Shows reasonable analysis with some	Limited analysis; weak reasoning	No analytical reasoning demonstrated

		solutions	justification		
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____